

4.0 C++ I/O System and File Handling

4.1 C++ Streams

4.2 Unformatted I/O operations

4.3 Formatted I/O operations

4.4 Manipulators

4.4 Opening and closing file

4.5 file modes

Streams

Programs often need to read data from devices such as a disk (where files are stored) and the keyboard.

It's also common that programs write data to files in a disk or to a computer display. Reading from and writing to these devices directly is not an easy task.

There are many low level technical details that need to be considered and each device may have its own specificities.

Therefore, the C++-standard library offers streams. A stream is an abstraction that represents a device, such as a file or keyboard, on which input (i.e. read) and output (i.e. write) operations are performed in a much simpler way.

C++ programs do not read directly from, or write directly to, a device.

C++-programs read from and write to streams.

A stream is a sequence of bytes or character.

Streams hides the programmer from all low level technical intricacies of the different devices.

It represents a device on which input and output operations are performed.

It can be represented as a source or destination of characters of indefinite length.

It is generally associated to a physical source or destination of characters like a disk file, keyboard or console.

C++ provides a standard iostream library to operate with streams.

input and output stream :-

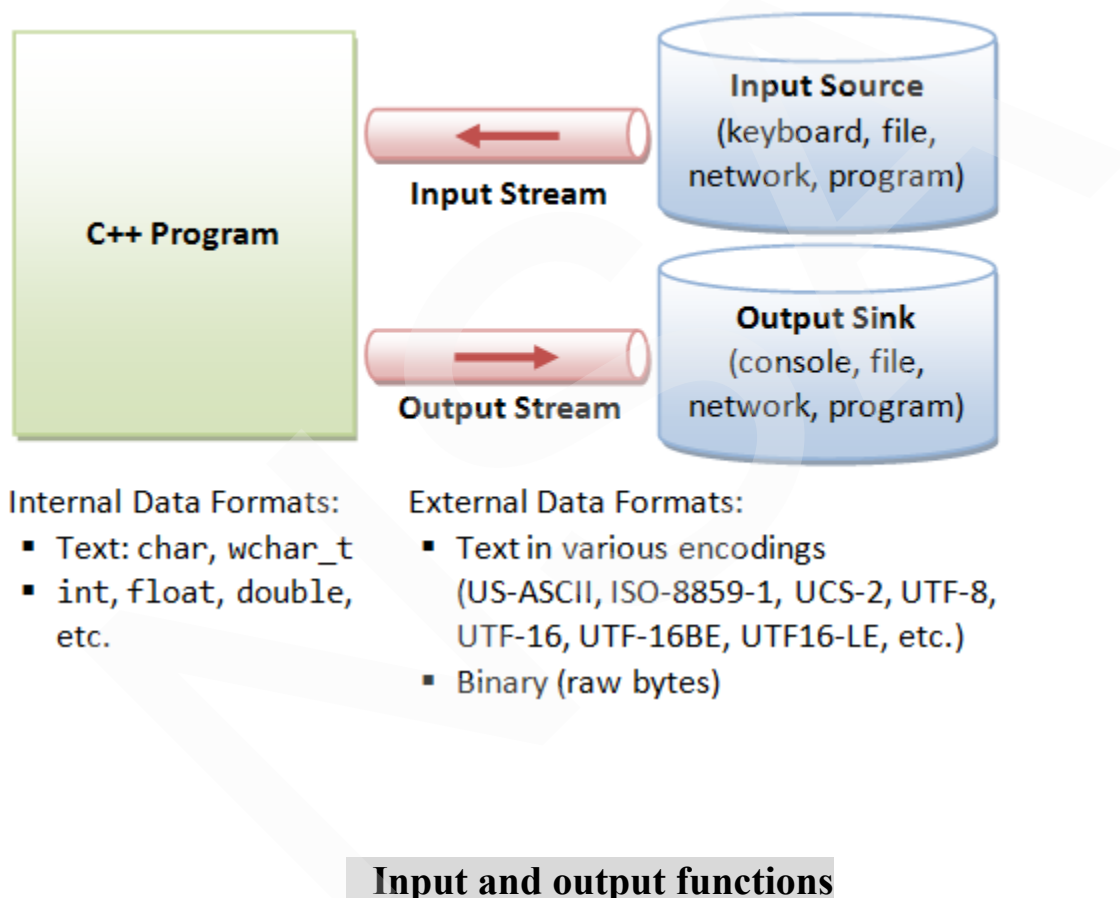
Input Stream: If the direction of flow of bytes is from the device (for example, Keyboard) to the main memory then this process is called input.

Output Stream: If the direction of flow of bytes is opposite, i.e. from main memory to device(display screen) then this process is called output.

We have been using two specific streams in our programs, so far: `std::cin` and `std::cout`. A stream is connected to a device such as the keyboard.

A stream variable consists of

- a sequence of bytes, also known as buffer, where characters are placed and
- three bits that indicate the state of the stream.



Console input / output function take input from standard input devices and compute and give output to standard output device.

Generally, keyboard is standard input device and monitor is standard output device.

In case of C++ it uses streams to perform input and output operations in standard input output devices (keyboard and monitor).

A stream is an object which can either insert or extract the character from it.

The standard C++ library is iostream and standard input / output functions in C++ are: 1. cin 2. cout There are mainly two types of console I/O operations form:

1. Unformatted console input output
2. Formatted console input output

1) Unformatted console input output operations :-

These input / output operations are in unformatted mode. The following are operations of unformatted console input / output operations:

A) void get()

It is a method of cin object used to input a single character from keyboard. But its main property is that it allows wide spaces and newline character.

Syntax:

```
char c=cin.get();
```

Example:

```
#include<iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
char c=cin.get();
```

```
cout<<c<<endl;
```

```
return 0;
```

```
}
```

Output

I

I

B) void put()

It is a method of cout object and it is used to print the specified character on the

screen or monitor.

Syntax:

```
cout.put(variable / character);
```

Example:

```
#include using namespace std;

int main()
{
    char c=cin.get();
    cout.put(c); //Here it prints the value of variable c;
    cout.put('c'); //Here it prints the character 'c'; return 0;
}
```

Output

I

Ic

C) getline(char *buffer,int size)

This is a method of cin object and it is used to input a string with multiple spaces.

The getline()function reads a whole line of the text and ends with newline character.

This function can be invoked by, cin.getline(line,size);

Syntax:

```
char x[30];

cin.getline(x,30); // It takes 30 charcters as input;
```

Example:

```
#include <iostream>
using namespace std;

int main()
{
    cout<<"Enter name :";
    char c[10];
    cin.getline(c,10); //It takes 10 charcters as input;
    cout<<c<<endl;
```

```
return 0
```

```
}
```

Output Enter name :

Divyanshu

Divyanshu

E) cin

It is the method to take input any variable / character / string.

Syntax: cin>>variable / character / String / ;

Example :-

```
#include<iostream>
```

```
using namespace std;
```

```
int main() {
```

```
int num;
```

```
cout<<"Enter Number";
```

```
cin>>num; //Inputs a variable;
```

```
return 0;
```

```
}
```

Output :

Enter Number 07

F) cout

This method is used to print variable / string / character.

Syntax: cout<< variable / character / string;

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int num=100;
```

```
cout<<"Number is "<<num<<endl; //Prints value of variable;
```

```
return 0;
```

```
}
```

Output :

Number is 100

2) Formatted console input output operations

In formatted console input output operations we use following functions to make output in perfect alignment.

In industrial programming all the output should be perfectly formatted due to this reason C++ provides many functions to convert any file into perfect aligned format.

These functions are available in header file `<iomanip>` refers input output manipulations.

A) width(n)

This function is used to set width of the output.

Syntax:

```
cout<<setw(int n);
```

Example:

```
#include<iostream>
```

```
#include<iomanip>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int x=10;
```

```
cout<<setw(20)<<variable;
```

```
return 0;
```

```
}
```

Output :- 10

B) fill(char)

This function is used to fill specified character at unused space.

Syntax:

```
cout<<setfill('character')<<variable;
```

```
#include<iostream>
```

```
#include<iomanip>
```

```
using namespace std;
int main()
{
int x=10;
cout<<setw(20);
cout<<setfill('#')<<x;
return 0;
}
```

Output

```
#####10
```

D) precision(n)

This method is used for setting the floating point of the output.

Syntax:

```
cout<<setprecision('int n')<<variable;
```

Example:

```
#include<iostream>
#include<iomanip>
using namespace std;
int main()
{
float x=10.12345;
cout<<setprecision(5)<<x;
return 0;
}
```

Output 10.123

E) setflag(arg 1, arg,2)

This function is used for setting format flags for output.

Syntax: setiosflags(argument 1, argument 2);

F) unsetflag(arg 2)

This function is used to reset set flags for output.

Syntax: resetiosflags(argument 2);

G) **setbase(arg)**

This function is used to set basefield of the flag.

Syntax: `setbase(argument);`

C++ Manipulators

In C++ programming, language manipulators are used in the formatting of output. These are helpful in modifying the input and the output stream. They make use of the insertion and extraction operators to modify the output.

Here's a list of a few manipulators:

Manipulators	Description
<code>endl</code>	It is used to enter a new line with a flush.
<code>setw(a)</code>	It is used to specify the width of the output.
<code>setprecision(a)</code>	It is used to set the precision of floating-point values.
<code>setbase(a)</code>	It is used to set the base value of a numerical number.

Note that we use the header file *iomanip* for some of the manipulators.

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
    float PI = 3.14;
    int num = 100;
    cout << "Entering a new line." << endl;
    cout << setw(10) << "Output" << endl;
    cout << setprecision(10) << PI << endl;
```



```
cout << setbase(16) << num << endl; //sets base to 16
}
```

Output:

Entering a new line.

Output
3.140000105

64

File Handling in C++

Files are primarily used to store data permanently, unlike memory storage which is volatile.

C++ provides a comprehensive set of classes and methods in its standard library, mainly the `<fstream>` header file, to manage and manipulate files.

This enables operations on text files (text mode) and binary files (binary mode).

The primary classes for file handling in C++ are:

- **ofstream:** Represents the output file stream and is used to create and write to files.
- **ifstream:** Represents the input file stream and is used to read from files.
- **fstream:** Represents the file stream and can be used for both reading from and writing to files.

Steps of File Handling:

1.Open a File: Before performing any operation on a file, it must be opened. In C++, we use the `open()` method.

```
ofstream outfile;  
outfile.open("example.txt");
```

2.Perform Operation: Once opened, you can perform various operations like reading from or writing to the file.

- Writing: `outfile << "Writing to file." << endl;`
- Reading: `infile >> variable;`

3. Close the File: After completing operations on the file, it's essential to close it using the `close()` method. Closing the file releases the resources that were allocated for that file during the

```
operation.  
outfile.close();
```

C++ Streams for File Handling

In C++, file handling operations are achieved through streams. A stream is an abstract entity where a program can either insert or extract data. It acts as an intermediary between the program and the input/output devices. When dealing with files, C++ uses specific stream classes, which manage and manipulate files.

Classes for File Stream Or File I/O Functions

These are some useful classes for working with files in C++

1.fstream -(file stream class)

A combination of ofstream and ifstream: creates, reads, and writes to files

Example :-

```
fstream ioFile;  
ioFile.open("data.txt", ios::in | ios::out); // Open for input and output
```

2 ofstream - (Output file stream)

creates and writes to files

Example :-

```
ofstream outFile;  
outFile.open("example.txt");
```

3 ifstream - (Input file stream)

reads from files

Example :-

```
ifstream inFile;  
inFile.open("sample.txt");
```

The fstream library allows us to handle files. So, to be able to use files in a program, one must include the **<fstream>** header file.

Opening a File

To start working with the file, first, we need to open it in our program. We can either open our file with the constructor provided by the file I/O classes or call the open method on the stream object.

Opening Modes

There are several modes we can specify when opening a file. These modes correspond to various controls given to stream objects during file handling in c++.

Mode	Syntax	Description
Read	ios::in	Opens file for reading purpose.
Write	ios::out	Opens a file for writing purposes.
Binary	ios::binary	All operations will be performed in binary mode.
Truncate before Open	ios::trunc	If the file already exists, all content will be removed immediately.
Append	ios::app	All provided data will be appended in the associated file.
At End	ios::ate	It opens the file and moves the read/write control at the End of the File. The basic difference between the ios::app and this one is that the former will always start writing from the end, but we can seek any particular position with this one.

A. Opening a file

In order to work with files in C++, we will first have to open it. Primarily, there are two ways to open a file:

- Using the constructor
- Using the member function open() of the class

A file could be opened for a number of uses. It could be to write to it or to read from it.

Consider an example that demonstrates the opening of a file using a constructor.

```
#include <iostream>
#include <fstream>
```

```
using namespace std;
```

```
int main()
{
    ofstream out("example.txt");
    return 0;
}
```

The file example.txt gets created if it's not already there in the system and opened. Object out gets created of the type ofstream, which means it could only be used to write into the opened file.

This is how we use the constructor ofstream to open a file. Another example demonstrates the use of the ifstream constructor.

```
#include <iostream>
#include <fstream>

using namespace std;

int main()
{
    ifstream in("example.txt");
    return 0;
}
```

For this to work, a file named example.txt must already be created and present in the same folder as that of the program. Object in gets created of the type ifstream, which means it could only be used to read from the file.

B. Closing a file

When working with C++, closing open files is considered a good practice. A programmer often makes the mistake of not closing an open file. This becomes crucial because files do not automatically get closed after a program uses them. The closing has to be done manually. Here are some reasons why it is necessary to close the file,

- The data might be in the buffer after the write operation, so closing a file will cause the data to be written in the file immediately.
- When you need to use the same stream with another file, it is a good practice to close the previous file.
- To free the resources held by the file.

To close a file, we have to use the close method. This is how we use them in C++.

Syntax:

```
file_objectname.close();
```

C. Writing to a file

Writing to a file is as easy as printing any other stuff in C++. It is very similar to what we used to do when we had to print an output in the terminal. In order to write to a file, we use the insertion operator (<<). First, we create an object of the type ofstream and pass the name of the file along with its extension to the method. And then, use the extraction operator to write stuff in the file fed to the object.

Consider an example demonstrating how we write to a file.

Example:

```
#include <iostream>
#include <fstream>

using namespace std;

int main()
{
    string str = "Welcome_To_Our_Group!";
    ofstream out("example.txt");
    out << str;
    return 0;
}
```

Output in the example.txt file:

```
Welcome_To_Our_Group!
```

D. Reading a file

Reading from a file is as easy as reading any other stuff as an input in C++. It is very similar to what we used to do when we had to read input from the terminal. In order to read from a file, we use the extraction operator (>>). First, we create an object of the type ifstream and pass the name of the file along with its extension to the method. And then, use the extraction operator to read stuff from the file fed to the object.

Consider an example demonstrating how we read from a file.

Example:

```
#include <iostream>
```

```
#include <fstream>

using namespace std;

int main()
{
    string str;
    ifstream in("example.txt");
    in >> str;
    cout << str;
    return 0;
}
```

The file example.txt had "Welcome_To_CodeWithHarry!" as its content, hence the output:

```
Welcome_To_Our_Group!
```